PRACTICEMINIPROJECT

December 2, 2025

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import plotly.express as px
import seaborn as sns
```

[2]:
```
df=pd.read_csv('creditcard.csv')
```

[3]:
```
df
```

[3]:
```
            Time         V1         V2        V3        V4        V5  \
0            0.0  -1.359807  -0.072781  2.536347  1.378155 -0.338321
1            0.0   1.191857   0.266151  0.166480  0.448154  0.060018
2            1.0  -1.358354  -1.340163  1.773209  0.379780 -0.503198
3            1.0  -0.966272  -0.185226  1.792993 -0.863291 -0.010309
4            2.0  -1.158233   0.877737  1.548718  0.403034 -0.407193
...          ...        ...        ...       ...       ...       ...
284802  172786.0 -11.881118  10.071785 -9.834783 -2.066656 -5.364473
284803  172787.0  -0.732789  -0.055080  2.035030 -0.738589  0.868229
284804  172788.0   1.919565  -0.301254 -3.249640 -0.557828  2.630515
284805  172788.0  -0.240440   0.530483  0.702510  0.689799 -0.377961
284806  172792.0  -0.533413  -0.189733  0.703337 -0.506271 -0.012546

                V6        V7        V8        V9  ...       V21       V22  \
0         0.462388  0.239599  0.098698  0.363787  ... -0.018307  0.277838
1        -0.082361 -0.078803  0.085102 -0.255425  ... -0.225775 -0.638672
2         1.800499  0.791461  0.247676 -1.514654  ...  0.247998  0.771679
3         1.247203  0.237609  0.377436 -1.387024  ... -0.108300  0.005274
4         0.095921  0.592941 -0.270533  0.817739  ... -0.009431  0.798278
...            ...       ...       ...       ...  ...       ...       ...
284802 -2.606837 -4.918215  7.305334  1.914428  ...  0.213454  0.111864
284803  1.058415  0.024330  0.294869  0.584800  ...  0.214205  0.924384
284804  3.031260 -0.296827  0.708417  0.432454  ...  0.232045  0.578229
284805  0.623708 -0.686180  0.679145  0.392087  ...  0.265245  0.800049
284806 -0.649617  1.577006 -0.414650  0.486180  ...  0.261057  0.643078

                V23       V24       V25       V26       V27       V28  Amount  \
0         -0.110474  0.066928  0.128539 -0.189115  0.133558 -0.021053  149.62
1          0.101288 -0.339846  0.167170  0.125895 -0.008983  0.014724    2.69
2          0.909412 -0.689281 -0.327642 -0.139097 -0.055353 -0.059752  378.66
3         -0.190321 -1.175575  0.647376 -0.221929  0.062723  0.061458  123.50
4         -0.137458  0.141267 -0.206010  0.502292  0.219422  0.215153   69.99
...             ...       ...       ...       ...       ...       ...     ...
```

```
284802  1.014480 -0.509348  1.436807  0.250034  0.943651  0.823731    0.77
284803  0.012463 -1.016226 -0.606624 -0.395255  0.068472 -0.053527   24.79
284804 -0.037501  0.640134  0.265745 -0.087371  0.004455 -0.026561   67.88
284805 -0.163298  0.123205 -0.569159  0.546668  0.108821  0.104533   10.00
284806  0.376777  0.008797 -0.473649 -0.818267 -0.002415  0.013649  217.00

        Class
0           0
1           0
2           0
3           0
4           0
...       ...
284802      0
284803      0
284804      0
284805      0
284806      0

[284807 rows x 31 columns]
```

[4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
```

```
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

[5]:
```
df.isnull().sum()
```

[5]:
```
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
dtype: int64
```

[6]:
```
df.describe(include='all')
```

[6]:

	Time	V1	V2	V3	V4	\
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	
mean	94813.859575	1.175161e-15	3.384974e-16	-1.379537e-15	2.094852e-15	
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	

```
                 V5            V6            V7            V8            V9  \
count  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05
mean   1.021879e-15  1.494498e-15 -5.620335e-16  1.149614e-16 -2.414189e-15
std    1.380247e+00  1.332271e+00  1.237094e+00  1.194353e+00  1.098632e+00
min   -1.137433e+02 -2.616051e+01 -4.355724e+01 -7.321672e+01 -1.343407e+01
25%   -6.915971e-01 -7.682956e-01 -5.540759e-01 -2.086297e-01 -6.430976e-01
50%   -5.433583e-02 -2.741871e-01  4.010308e-02  2.235804e-02 -5.142873e-02
75%    6.119264e-01  3.985649e-01  5.704361e-01  3.273459e-01  5.971390e-01
max    3.480167e+01  7.330163e+01  1.205895e+02  2.000721e+01  1.559499e+01

       ...           V21           V22           V23           V24  \
count  ...  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05
mean   ...  1.628620e-16 -3.576577e-16  2.618565e-16  4.473914e-15
std    ...  7.345240e-01  7.257016e-01  6.244603e-01  6.056471e-01
min    ... -3.483038e+01 -1.093314e+01 -4.480774e+01 -2.836627e+00
25%    ... -2.283949e-01 -5.423504e-01 -1.618463e-01 -3.545861e-01
50%    ... -2.945017e-02  6.781943e-03 -1.119293e-02  4.097606e-02
75%    ...  1.863772e-01  5.285536e-01  1.476421e-01  4.395266e-01
max    ...  2.720284e+01  1.050309e+01  2.252841e+01  4.584549e+00

                V25           V26           V27           V28         Amount  \
count  2.848070e+05  2.848070e+05  2.848070e+05  2.848070e+05  284807.000000
mean   5.109395e-16  1.686100e-15 -3.661401e-16 -1.227452e-16      88.349619
std    5.212781e-01  4.822270e-01  4.036325e-01  3.300833e-01     250.120109
min   -1.029540e+01 -2.604551e+00 -2.256568e+01 -1.543008e+01       0.000000
25%   -3.171451e-01 -3.269839e-01 -7.083953e-02 -5.295979e-02       5.600000
50%    1.659350e-02 -5.213911e-02  1.342146e-03  1.124383e-02      22.000000
75%    3.507156e-01  2.409522e-01  9.104512e-02  7.827995e-02      77.165000
max    7.519589e+00  3.517346e+00  3.161220e+01  3.384781e+01   25691.160000

              Class
count  284807.000000
mean        0.001727
std         0.041527
min         0.000000
25%         0.000000
50%         0.000000
75%         0.000000
max         1.000000

[8 rows x 31 columns]
```

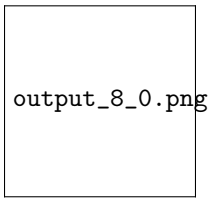
[7]:
```
df['Amount'].mean()
```

[7]:
```
np.float64(88.34961925093134)
```

[17]:
```
fraud_count
```

[17]:
```
Class
0    284315
1       492
Name: count, dtype: int64
```

[16]:
```
##Class Distribution (Fraud vs Non-Fraud)
fraud_count = df['Class'].value_counts()
plt.figure(figsize=(6,4))
sns.barplot(x=fraud_count.index, y=fraud_count.values,hue=fraud_count.index)
plt.title("Fraud vs Non-Fraud Transaction Count")
plt.xlabel("Class (0 = Non-Fraud, 1 = Fraud)")
plt.ylabel("Number of Transactions")
plt.show()
#value_counts()
##fraud_count.index → Class names (0 and 1)
##fraud_count.values → Their counts
##Tells us how many frauds and non-frauds exist..index
##Used to label the x-axis with 0 and 1..values
##Used to get the number of transactions for each class.
```



[ ]:
```
##5.Transaction Amount Distribution
plt.figure(figsize=(7,5))
sns.histplot(df['Amount'], bins=100, kde=True)
plt.title("Transaction Amount Distribution")
plt.show()
```

[ ]:
```
df.sort_values(by='Amount').value_counts(df[df['Amount']<=5000]['Amount'])
```

[ ]:
```
##6. Compare Fraud vs Non-Fraud Behaviour (EDA)
##6.1 Amount Comparison
plt.figure(figsize=(7,5))
sns.boxplot(x='Class', y='Amount', data=df)
plt.title("Fraud vs Non-Fraud Transaction Amount")
plt.show()
##Interpretation:Fraudulent transactions often have distinct patterns (either very␣
↪small or very large).
```

[ ]:
```
##6.2 Time-Based Behaviour
##Dataset has a Time column in seconds.
plt.figure(figsize=(10,5))
sns.kdeplot(df[df.Class==0]['Time'], label="Non-Fraud")
sns.kdeplot(df[df.Class==1]['Time'], label="Fraud")
plt.legend()
plt.title("Fraud vs Non-Fraud Transaction Time Pattern")
plt.show()
##df[df['Class']==1]['Amount']**
##Select the *Amount* column for fraud transactions.
##df[df['Class']==1]['Amount']**
##Select the *Amount* column for fraud transactions.
```

[ ]:
```
print("non-fraud","\n",df[df['Class']==0]['Time'],"\n","fraud","\n",df[df['Class']==1]['Time'])
```

[ ]:
```
##Correlation Study (Find Important Fraud Indicators)
plt.figure(figsize=(15,10))
sns.heatmap(df.corr(), cmap='coolwarm', center=0)
plt.title("Correlation Matrix")
plt.show()
```

[ ]:
```
##Money Flow Visualization (Using Plotly)
##This shows how money moves across amounts.
fig = px.histogram(df, x="Amount", color="Class", nbins=50, title="Money Flow: Fraud␣
↪vs Non-Fraud")
fig.show()
```

[ ]:
```
## Identify Red Flags Using EDA Only
##9.1 Unusual High Amount Transactions
high_amount = df[df['Amount'] > df['Amount'].quantile(0.99)]
high_amount[high_amount['Class'] == 1].head()
```

[ ]:
```
##9.2 Transactions in Rare Time Windows
rare_times = df[(df["Time"] < 500) | (df["Time"] > 85000)]
rare_times[rare_times["Class"] == 1].head()
```

[ ]:
```
##9.3 Top features linked to fraud
fraud_corr = df[df['Class'] == 1].corr()['Class'].sort_values(ascending=False)
fraud_corr.head(10)
```

[ ]:
```
##PCA Visualization (2D Plot)
##Helpful to show clusters of fraud.
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
# Scale data
X = df.drop('Class', axis=1)
X = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
pca_comp = pca.fit_transform(X)
df_pca = pd.DataFrame({"PC1": pca_comp[:,0], "PC2": pca_comp[:,1], "Class":␣
↪df["Class"]})
fig = px.scatter(df_pca, x='PC1', y='PC2', color='Class', title="PCA Visualization of␣
↪Fraud Patterns")
fig.show()
```

[ ]:
```
pip install scikit-learn
```

[ ]:
```
df['Time'].max()
```

[ ]:
```
df['Time'].min()
```

[ ]:
```
df[df['Time']>85000]['Time'].value_counts()
```

[9]:
```
corr = df.corr()
corr['Class'].sort_values(ascending=False)
```

[9]:
```
Class     1.000000
V11       0.154876
V4        0.133447
V2        0.091289
V21       0.040413
V19       0.034783
V20       0.020090
V8        0.019875
V27       0.017580
V28       0.009536
Amount    0.005632
V26       0.004455
V25       0.003308
V22       0.000805
V23      -0.002685
V15      -0.004223
V13      -0.004570
V24      -0.007221
Time     -0.012323
V6       -0.043643
V5       -0.094974
V9       -0.097733
V1       -0.101347
V18      -0.111485
V7       -0.187257
V3       -0.192961
V16      -0.196539
V10      -0.216883
V12      -0.260593
V14      -0.302544
V17      -0.326481
Name: Class, dtype: float64
```

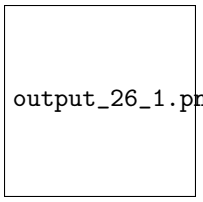
[10]:
```
df_fraud = df[df['Class'] == 1]
df_fraud[['V10','V12','V14']].describe()
```

[10]:

	V10	V12	V14
count	492.000000	492.000000	492.000000
mean	-5.676883	-6.259393	-6.971723
std	4.897341	4.654458	4.278940
min	-24.588262	-18.683715	-19.214325
25%	-7.756698	-8.688177	-9.692723
50%	-4.578825	-5.502530	-6.729720
75%	-2.614184	-2.974088	-4.282821
max	4.031435	1.375941	3.442422

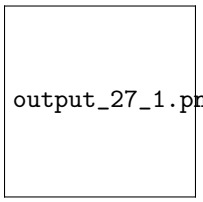
[11]:
```
plt.figure(figsize=(10,4))
sns.boxplot(x='Class', y='V10', data=df)
```

[11]:
```
<Axes: xlabel='Class', ylabel='V10'>
```



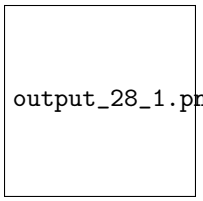
[13]:
```
plt.figure(figsize=(10,4))
sns.boxplot(x='Class', y='V14', data=df)
```

[13]:
```
<Axes: xlabel='Class', ylabel='V14'>
```



[14]:
```
plt.figure(figsize=(10,4))
sns.boxplot(x='Class', y='V12', data=df)
```

[14]:
```
<Axes: xlabel='Class', ylabel='V12'>
```



[ ]:
```

```